



King Abdulaziz University

Faculty of Computing & Information Technology Computer Science Department

Final Exam (July 29th, 2019)
Course: CPCS-223 (Analysis and Design of Algorithms)
Term: Summer 2019
Points: 100 Time: 120 minutes

SOLUTION

Student ID: _____

Student Name: _____

***Section: _____ **AA**

CLO	SO	Question	Max. Marks	Obt. Marks
05	B	1	10	
05	B	2	20	
11	B	3	20	
04	B	4	20	
08	B	5	20	
15	C	6	10	
		TOTAL	100	

Instructor:

☐ Dr Muhammad Umair

(Problem Solving)

(10 Marks)

ALGORITHM *ComparisonCountingSort*($A[0..n-1]$)

```
//Input: An array  $A[0..n - 1]$  of orderable elements
```

```
//Output: Array  $S[0..n-1]$  of  $A$ 's elements sorted in nondecreasing order
```

```

for  $i \leftarrow 0$  to  $n - 1$  do  $Count[i] \leftarrow 0$ 

```

for $i \leftarrow 0$ **to** $n - 2$ **do****for** $j \leftarrow i + 1$ **to** $n - 1$ **do**

```

if  $A[i] < A[j]$ 

```

$$Count[j] \leftarrow Count[j] + 1$$

```

else  $Count[i] \leftarrow Count[i] + 1$ 

```

```

for  $i \leftarrow 0$  to  $n - 1$  do  $S[Count[i]] \leftarrow A[i]$ 

```

A

Count

S

[illegible]

Using the given algorithm (**Johnson Trotter**), generate all permutations for **TKDB**

Perm#	Permutation
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	
20	
21	
22	
23	
24	

ALGORITHM *JohnsonTrotter*(n)

initialize the first permutation with $\overleftarrow{1} \overleftarrow{2} \dots \overleftarrow{n}$

while the last permutation has a mobile element **do**

 find its largest mobile element k

 swap k with the adjacent element k 's arrow points to

 reverse the direction of all the elements that are larger than k

 add the new permutation to the list

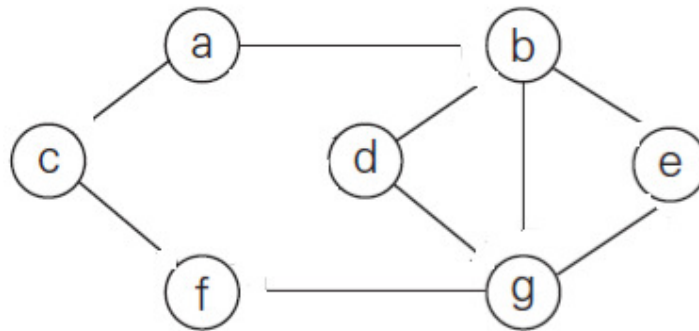
Rough Work Area:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

HVCLO – 11
Question No. 3

SO – B
(20) points

Find the maximum number of paths exist between any two vertices of the given algorithm using the “Transform and Conquer (Problem Reduction)” technique. **The path length is 4 (four)**. You are supposed to show all intermediate results too.



Solution:

Question 03 (Continue)

Question 03 (Continue)

CLO – 4

SO – B

Question No. 4

20 points

Solve the given recurrence using “**Substitution Method**”. All steps and calculation must be shown for getting full credit

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$$

Solution:

Compare the following two algorithms in terms of their performance (Complexity).

ALGORITHM *DistributionCountingSort*($A[0..n-1], l, u$)

Input: An array $A[0..n-1]$

Output: Array $S[0..n-1]$

for $j \leftarrow 0$ **to** $u-l$ **do** $D[j] \leftarrow 0$

for $i \leftarrow 0$ **to** $n-1$ **do** $D[A[i]-l] \leftarrow D[A[i]-l] + 1$

for $j \leftarrow 1$ **to** $u-l$ **do** $D[j] \leftarrow D[j-1] + D[j]$

for $i \leftarrow n-1$ **downto** 0 **do**

$j \leftarrow A[i]-l$

$S[D[j]-1] \leftarrow A[i]$

$D[j] \leftarrow D[j]-1$

return S

Solution:

$\Theta(n)$

ALGORITHM *ComparisonCountingSort*($A[0..n-1]$)

//Sorts an array by comparison counting

//Input: An array $A[0..n-1]$ of orderable elements

//Output: Array $S[0..n-1]$ of A 's elements sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n-1$ **do** $Count[i] \leftarrow 0$

for $i \leftarrow 0$ **to** $n-2$ **do**

for $j \leftarrow i+1$ **to** $n-1$ **do**

if $A[i] < A[j]$

$Count[j] \leftarrow Count[j] + 1$

else $Count[i] \leftarrow Count[i] + 1$

for $i \leftarrow 0$ **to** $n-1$ **do** $S[Count[i]] \leftarrow A[i]$

return S

Solution:

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\ &= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2} \\ &= \Theta(n^2) \end{aligned}$$

The **Dutch national flag problem** is to rearrange an array of characters R, W, and B (red, white, and blue are the colors of the Dutch national flag) so that all the R's come first, the W's come next, and the B's come last. **Design** a linear in-place algorithm for this problem.

Solution:

```
Algorithm DutchFlag( $A[0..n-1]$ )
//Sorts an array with values in a three-element set
//Input: An array  $A[0..n-1]$  of characters from {'R', 'W', 'B'}
//Output: Array  $A[0..n-1]$  in which all its  $R$  elements precede
//         all its  $W$  elements that precede all its  $B$  elements
 $r \leftarrow 0$ ;  $w \leftarrow 0$ ;  $b \leftarrow n-1$ 
while  $w \leq b$  do
    if  $A[w] = 'R'$ 
        swap( $A[r], A[w]$ );  $r \leftarrow r+1$ ;  $w \leftarrow w+1$ 
    else if  $A[w] = 'W'$ 
         $w \leftarrow w+1$ 
    else //  $A[w] = 'B'$ 
        swap( $A[w], A[b]$ );  $b \leftarrow b-1$ 
```

Extra Sheet for rough work